Atividade Prática – Desenvolvimento de um Sistema de Controle de Tarefas com JavaScript

Tema: Organização e gerenciamento de tarefas de um projeto
Linguagem: JavaScript
Ambiente: Node.js e Visual Studio Code
Padrão de modularização: ES Modules (ESM)
Execução: Terminal

1. Contextualização

Uma equipe de desenvolvimento de sistemas precisa organizar as atividades realizadas durante a construção de seus projetos. Para isso, é necessário acompanhar as tarefas que precisam ser executadas, identificar os responsáveis, definir prioridades e controlar o andamento de cada atividade.

Imagine que você foi contratado para desenvolver um pequeno sistema de controle de tarefas que será utilizado por uma equipe de desenvolvimento.

O sistema deverá permitir cadastrar tarefas, consultar as atividades registradas, atualizar seus status e gerar relatórios sobre o andamento do projeto.

O desenvolvimento deverá utilizar os conhecimentos de programação estudados durante as aulas, incluindo variáveis, operadores, estruturas condicionais, estruturas de repetição, arrays, objetos, funções e modularização.

Atenção: o projeto deverá ser desenvolvido utilizando o padrão ES Modules, com import e export.

---

2. Objetivo da atividade

Desenvolver um sistema de controle de tarefas executado pelo terminal, organizando o código em diferentes arquivos e utilizando funções para separar as responsabilidades de cada módulo.

Ao concluir a atividade, o sistema deverá permitir:

- Cadastrar novas tarefas.
- Listar todas as tarefas cadastradas.
- Buscar uma tarefa pelo seu identificador.
- Atualizar o status de uma tarefa.
- Consultar tarefas de acordo com seu status.
- Gerar relatórios sobre as atividades do projeto.

Os dados deverão ser armazenados em um array de objetos durante a execução do programa.

---

3. Organização obrigatória do projeto

Crie uma pasta chamada controle_tarefas.

Dentro dela, crie os seguintes arquivos:

controle_tarefas/

│

├── index.js

├── entrada.js

├── tarefas.js

├── relatorios.js

│

└── package.json

Cada arquivo deverá possuir uma responsabilidade específica.

Arquivo	**Responsabilidade**
index.js	Controlar o menu principal e chamar as funções dos demais módulos.
entrada.js	Centralizar as funções responsáveis por receber dados do usuário.
tarefas.js	Armazenar, cadastrar, listar, buscar e atualizar tarefas.
relatorios.js	Gerar informações e estatísticas sobre as tarefas.
package.json	Configurar o projeto para utilizar ES Modules.

Configuração do projeto

Crie o arquivo package.json com a configuração:

```
{
  "name": "controle-tarefas",
  "version": "1.0.0",
  "type": "module"
}
```

---

4. Modelo de dados: como uma tarefa deverá ser armazenada?

Cada tarefa cadastrada deverá ser representada por um objeto JavaScript.

O objeto deverá possuir as seguintes propriedades:

Propriedade	Descrição	Exemplo
id	Identificador único da tarefa.	1
titulo	Nome da atividade.	Criar tela de login
descricao	Detalhamento da atividade.	Desenvolver a interface de acesso.
responsavel	Pessoa responsável pela tarefa.	Carlos
prioridade	Grau de importância.	Alta
status	Situação atual da tarefa.	Pendente

Exemplo de objeto

```
const tarefa = {
   id: 1,
   titulo: "Criar tela de login",
   descricao: "Desenvolver a interface de acesso",
   responsavel: "Carlos",
   prioridade: "Alta",
   status: "Pendente"
};
```

Todas as tarefas deverão ser armazenadas em um array chamado tarefas.

Exemplo de como esse array poderá ficar:

```
const tarefas = [
   {
      id: 1,
      titulo: "Criar tela de login",
      descricao: "Desenvolver a interface de acesso",
      responsavel: "Carlos",
      prioridade: "Alta",
      status: "Pendente"
   },
   {
      id: 2,
```

```
    titulo: "Criar banco de dados",
    descricao: "Modelar as tabelas do sistema",
    responsavel: "Mariana",
    prioridade: "Média",
    status: "Em andamento"
  },
  {
    id: 3,
    titulo: "Documentar o projeto",
    descricao: "Criar documentação das funcionalidades",
    responsavel: "Pedro",
    prioridade: "Baixa",
    status: "Concluída"
  }
];
```

O exemplo apresenta três tarefas para demonstrar a estrutura. No sistema desenvolvido, os alunos deverão permitir que o próprio usuário cadastre novas tarefas.

Valores permitidos

Para manter um padrão de informações, o sistema deverá trabalhar com as seguintes opções:

Prioridades:

- Baixa
- Média
- Alta

Status:

- Pendente
- Em andamento
- Concluída

---

5. Desenvolvimento dos módulos

Agora que a estrutura do projeto e os dados foram definidos, desenvolva cada arquivo seguindo as orientações.

ETAPA 1 – Arquivo entrada.js

Objetivo: criar funções que facilitem a entrada de informações pelo usuário.

Esse arquivo deverá importar a biblioteca readline-sync e disponibilizar duas funções.

Função 1: perguntarTexto(mensagem)

Responsabilidade:

- Receber uma mensagem como parâmetro.
- Apresentar a mensagem no terminal.
- Aguardar o usuário digitar uma informação.
- Retornar o texto digitado.

Exemplo de utilização:

let nome = perguntarTexto("Digite seu nome: ");

Função 2: perguntarNumero(mensagem)

Responsabilidade:

- Receber uma mensagem como parâmetro.
- Solicitar uma informação numérica.
- Converter a resposta para o tipo Number.
- Retornar o número informado.

Exemplo de utilização:

let idade = perguntarNumero("Digite sua idade: ");

Requisito de modularização: as duas funções deverão ser exportadas utilizando export, permitindo que outros arquivos do projeto possam utilizá-las.

ETAPA 2 – Arquivo tarefas.js

Objetivo: concentrar as funcionalidades relacionadas ao gerenciamento das tarefas.

Esse arquivo deverá criar o array tarefas e implementar as funções responsáveis por manipular seus dados.

Implemente obrigatoriamente as seguintes funcionalidades:

1. cadastrarTarefa()

Responsabilidade:

- Solicitar o título da tarefa.

- Solicitar a descrição.
- Solicitar o nome do responsável.
- Solicitar a prioridade.
- Criar um objeto contendo as informações.
- Gerar um identificador para a nova tarefa.
- Definir automaticamente o status inicial como Pendente.
- Adicionar o objeto ao array tarefas.
- Exibir uma mensagem confirmando o cadastro.

Regras:

- O título não poderá ficar vazio.
- A descrição não poderá ficar vazia.
- O responsável não poderá ficar vazio.
- A prioridade deverá ser uma das opções permitidas: Baixa, Média ou Alta.
- O identificador deverá ser diferente dos identificadores das demais tarefas.

2. listarTarefas()

Responsabilidade:

- Verificar se existem tarefas cadastradas.
- Caso não existam, apresentar uma mensagem informativa.
- Caso existam, percorrer o array utilizando uma estrutura de repetição.
- Exibir todas as informações de cada tarefa.

A listagem deverá apresentar:

- Identificador.
- Título.
- Descrição.
- Responsável.
- Prioridade.
- Status.

3. buscarTarefaPorId(id)

Responsabilidade:

- Receber um identificador como parâmetro.
- Procurar no array uma tarefa que possua aquele identificador.

- Retornar o objeto encontrado.
- Caso não encontre, retornar null.

Utilize o método find() ou uma estrutura de repetição com uma condição para realizar a busca.

4. atualizarStatus(id)

Responsabilidade:

- Solicitar o identificador da tarefa que será atualizada.
- Procurar a tarefa correspondente.
- Caso ela exista, apresentar as opções de status.
- Solicitar o novo status.
- Atualizar a propriedade status do objeto.
- Apresentar uma mensagem confirmando a atualização.

Regras:

- Somente poderão ser utilizados os status: Pendente, Em andamento e Concluída.
- Se o identificador não existir, apresentar uma mensagem informando que a tarefa não foi encontrada.
- Se o status informado for inválido, não realizar a atualização.

5. listarTarefasPorStatus(status)

Responsabilidade:

- Receber um status como parâmetro.
- Localizar todas as tarefas que possuem aquele status.
- Exibir as tarefas encontradas.
- Caso não existam tarefas naquela situação, apresentar uma mensagem.

Utilize o método filter() ou uma estrutura de repetição com condição.

Requisito de modularização: as funções deverão ser exportadas para que possam ser utilizadas pelo arquivo principal.

ETAPA 3 – Arquivo relatorios.js

Objetivo: apresentar informações que permitam acompanhar o andamento das atividades cadastradas.

Esse módulo deverá importar o array tarefas do arquivo tarefas.js.

Implemente as seguintes funções:

1. relatorioGeral()

Responsabilidade:

- Informar a quantidade total de tarefas cadastradas.
- Informar quantas tarefas estão pendentes.
- Informar quantas tarefas estão em andamento.
- Informar quantas tarefas foram concluídas.

Para isso, utilize estruturas de repetição e condicionais para percorrer o array e contar as tarefas de acordo com seu status.

2. relatorioPrioridades()

Responsabilidade:

- Contar quantas tarefas possuem prioridade Baixa.
- Contar quantas tarefas possuem prioridade Média.
- Contar quantas tarefas possuem prioridade Alta.
- Apresentar os resultados no terminal.

3. relatorioResponsaveis()

Responsabilidade:

- Percorrer as tarefas cadastradas.
- Apresentar o título e o responsável de cada tarefa.
- Permitir visualizar quais atividades estão atribuídas a cada integrante.

Requisito de modularização: as funções de relatório deverão ser exportadas para utilização no arquivo principal.

---

6. ETAPA 4 – Arquivo index.js: menu principal

Objetivo: controlar o funcionamento do sistema.

Esse será o arquivo responsável por reunir as funcionalidades dos demais módulos.

Ele deverá importar:

- As funções de entrada do arquivo entrada.js.
- As funções de gerenciamento do arquivo tarefas.js.
- As funções de relatório do arquivo relatorios.js.

Utilize a sintaxe import e export do padrão ES Modules.

Menu obrigatório

O programa deverá apresentar o seguinte menu:

```
==================================
    SISTEMA DE CONTROLE
```

DE TAREFAS

==========

1 - Cadastrar tarefa

2 - Listar todas as tarefas

3 - Buscar tarefa por ID

4 - Atualizar status de tarefa

5 - Listar tarefas por status

6 - Relatório geral

7 - Relatório por prioridade

8 - Relatório de responsáveis

0 - Sair

Escolha uma opção:

Funcionamento das opções

Opção	**Funcionalidade**
1	Chamar a função de cadastro de tarefas.
2	Chamar a função que lista todas as tarefas.
3	Solicitar um ID e chamar a função de busca.
4	Solicitar um ID e chamar a função de atualização de status.
5	Solicitar um status e chamar a função de listagem por status.
6	Apresentar o relatório geral.
7	Apresentar o relatório por prioridade.
8	Apresentar o relatório de responsáveis.
0	Encerrar o programa.

Regras para o menu

- O menu deverá ser apresentado repetidamente até que o usuário escolha a opção 0.
- Sugestão: Utilize uma estrutura de repetição do...while ou while.

- Utilize switch/case para identificar a opção escolhida.
- Utilize break para encerrar cada caso do switch.
- Utilize default para informar quando uma opção inválida for selecionada.
- Ao selecionar a opção 0, apresentar uma mensagem de encerramento.

---

7. Requisitos técnicos obrigatórios

Durante o desenvolvimento, observe os seguintes requisitos:

Conhecimento	**Como deverá ser utilizado**
Variáveis e constantes	Armazenar dados, opções e resultados das operações.
Tipos de dados	Trabalhar com textos e números.
Operadores	Realizar comparações e operações necessárias.
Condicionais	Validar informações e verificar situações.
Repetições	Controlar o menu e percorrer os arrays.
Arrays	Armazenar as tarefas cadastradas.
Objetos	Representar cada tarefa e suas propriedades.
Funções	Separar as funcionalidades do sistema.
Parâmetros	Enviar informações entre as funções.
return	Retornar resultados das operações e buscas.
Modularização	Organizar o código em diferentes arquivos.
ES Modules	Utilizar import e export para compartilhar funções e dados.

Restrições:

- Não utilizar require() ou module.exports.
- Não concentrar todo o código no arquivo index.js.
- Não deixar de utilizar os módulos definidos na estrutura do projeto.

---

8. Testes obrigatórios

Depois de implementar o sistema, realize os seguintes testes:

Teste 1 – Cadastro

- Cadastre pelo menos quatro tarefas.
- Utilize responsáveis diferentes.
- Distribua as prioridades entre Baixa, Média e Alta.

Teste 2 – Listagem

- Utilize a opção de listar todas as tarefas.
- Verifique se todas as informações aparecem corretamente.

Teste 3 – Busca

- Busque uma tarefa utilizando um ID existente.
- Depois, tente buscar um ID que não exista.
- Verifique se o sistema apresenta as mensagens adequadas.

Teste 4 – Atualização

- Atualize o status de uma tarefa para Em andamento.
- Atualize outra tarefa para Concluída.
- Tente atualizar uma tarefa com um ID inexistente.

Teste 5 – Filtro por status

- Liste somente as tarefas pendentes.
- Liste somente as tarefas em andamento.
- Liste somente as tarefas concluídas.

Teste 6 – Relatórios

- Execute o relatório geral.
- Execute o relatório de prioridades.
- Execute o relatório de responsáveis.
- Verifique se os números apresentados correspondem aos dados cadastrados.

Teste 7 – Menu

- Informe uma opção inválida.
- Verifique se o sistema apresenta uma mensagem de erro.
- Selecione a opção 0 e verifique se o programa encerra corretamente.

Resultado esperado: um pequeno sistema de controle de tarefas, desenvolvido em JavaScript, organizado em módulos e executado pelo terminal, demonstrando a aplicação dos conceitos de programação estudados durante as aulas.